Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Anonymous

Abstract

Checking a stored proof can avoid its original search, while checking a computational certificate can repeat the calculation that justifies it. We measure a pinned AI-generated Lean library and build ProofServe, an original-kernel check/audit tool. Matched target windows retain 80.28–82.35% of source-compilation wall time for two computational certificates, versus 0.46–6.54% for two ordinary controls. Across 31 module-process pairs, the median checker/compiler CPU ratio is 0.451, with overlapping source-proxy classes. 162,170 of 163,096 modules (99.43%) completed, using 168.956 P / 302.399 E CPU-hours; 926 were deferred. These are separate observed-work lower bounds; the missing tail is not imputed. The full-frame import graph omits 19.78% of source files and 33.30% of lexical kernel-decision sites. ProofServe integrates dependency-aware parallel replay, restartable segments and scoped receipts. From one to eight workers, Nat.Basic/Grid/Barrier wall speedups are 1.70/2.14/1.73×, with higher CPU. By sharing kernel-checked theorem bodies, default mode can accept crafted artifacts that official checking rejects in their original import context. Strict isolation costs 4.23× wall on Nat.Basic at eight workers.

1 Introduction

Proof-embedded computation can preserve expensive work across the boundary between producing and checking an AI-generated proof. Lean validates elaborated terms rather than repeating tactic search, but definitional equality can still evaluate a program inside a term. Users therefore need both a cost model and a precise account of what a successful project check covers.

Existing work already establishes the value of independent checking. Lean 4.35.0 adds `lake check` to export and recheck default targets, following the FRO's external-checking agenda [8, 6]. A public report on the same OpenAI library performs a complete cloud build and extensive independent checking [10]. It proposes kernel computation as an explanation for expensive checks from three source examples. Our empirical increment is a matched module-level measurement of that explanation, an observed dependency prefix with explicit condition-dependent lower bounds, and a coverage analysis at a different corpus revision. Our systems contribution is ProofServe's original-kernel workflow and auditable receipts, evaluated through its release-form interface. The four questions follow these boundaries:

RQ1 How do checking costs differ between stored mathematical proofs and computational certificates?

RQ2 What work is observed in the measured dependency prefix?

RQ3 How does parallel original-kernel checking trade wall time, CPU work and additional trusted code?

RQ4 What do default builds, repeated verdicts and axiom receipts establish about verification coverage?

2 Method and workload

A per-module observed ledger and matched checking windows separate measured library work from selected-case checking behavior. Our primary corpus is the published OpenAI mathematics repository [11], pinned at `4eaee875`: 163,096 executable modules plus one configuration unit, Lean 4.34.1 and Mathlib `d13f23b7`. The original 42 compilation observations and their strata remain historical calibration; they never fill the unmeasured tail.

Source cost includes a module's imports, tactics, kernel checking and artifact emission. Separate prerequisite preparation, C compilation and build-driver work are outside this clock. CPU means user plus system time. Whole-process replay and incremental target windows are distinct measurements; Figure 1 labels both. Reported sample and release timings use CPUs 0–7 of an i9-14900K, one task at a time, retaining file caches. Lean runs omit `-M`, including `-M0`; external memory protection is recorded separately.

The public package `data/openai-math-4eaee875/` records sample identities, raw-cost provenance, clocks and estimates. This paper pins its original-sample summary to `1cc14067`. Historical replay measurements explain cost boundaries; system performance must come from committed release-tool results in `checker/results/` and `data/external-projects/`.

Compilation uses eight P and fourteen E cores and unchanged `-j8` commands. Segment 1 has fixed 22-slot admission with reserve drains/exclusive

retry. Segment 2 uses memory/headroom admission and defers unfinished `PadicHodge`, `Ramsey.CycleClique.Construction`, `BigCrystalline` and other interrupted families plus dependents. These are *deferred by user*, not failed or failure-blocked; earlier completed records are retained. The second rule continues across a retired boundary; intervals and gaps stay distinct. The total-instruction gate failed: median 1.33% and P95 3.15% exceeded unchanged 1%/3% criteria. User-mode comparison was chosen afterwards; P/E CPU is unconverted.

3 Two costs and observed library work

3.1 Computational certificates retain work

Matched target windows show expensive replay for computational certificates and cheap replay for ordinary controls. In Figure 1(b), Grid and Amplitudes retain 80.28% and 82.35% of their source compiler's wall time. Their replay components are 255.489 and 110.026 seconds; almost all of each is inside the kernel replay call. Mathlib Nat.Basic and NearPin retain 6.54% and 0.46%. Each target component reads, maps and checks that target after dependency preparation; each denominator is that module's compiler process. These shared boundaries quantify the expensive kernel replay anticipated by the report's computation hypothesis in the selected cases [10].

The mechanism is visible in the proof boundary. A tactic can search for a mathematical derivation and emit a term that records its successful steps. Replaying that term validates those steps without reconstructing the search. A finite certificate justified through `of_decide_eq_true` instead asks the kernel to validate the decision procedure's result through reduction. The compact wrapper can therefore retain evaluation over a large finite object. Our target measurement holds the declared input and original checking operation fixed and locates that retained cost in the replay window. Its practical consequence is that source length and a cheap closure average are weak planning signals for an individual certificate.

The broader paired panel shows variation rather than a clean binary split. Across 31 original modules, checker/compiler CPU has median 0.451: 0.391 for the 14 unmarked and 0.498 for the 17 marked modules. An explicit comment/string-masked `decide +kernel` site is a reproducible proxy for computation, while ordinary `decide` can also encode substantial evaluation [2, 5]. For example, the unmarked Ramsey K10 certificate uses 356.44 checker CPU-seconds against 1,725.13 source CPU-seconds (20.66%). The dependency-ordered Ramsey prefix covers 190 of 444 targets; 20 sampled modules have a 35.91% aggregate ratio.

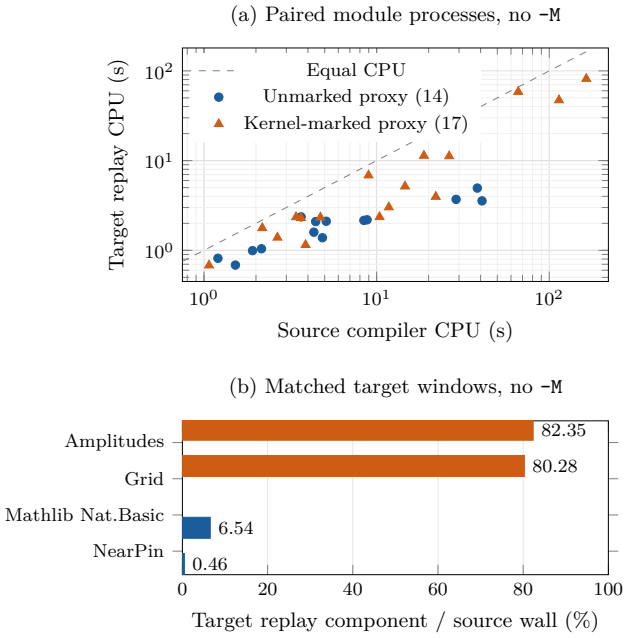


Figure 1: Two views of checking cost, all without `-M`. (a) Thirty-one complete module-process pairs; colors are source-marker proxies, whose distributions overlap. Imports are trusted in target replay. (b) Ordinary controls (blue) and computational certificates (orange), using matched target read/map/replay wall windows after dependency preparation. CPU in (a) and wall time in (b) have separate denominators. Design implication: expose the checking stage and amortize preparation; a closure average can hide an expensive certificate.

Whole closures dilute expensive targets with cheaper dependencies. Checkpointed Falconer checking costs 2,038.44/32,292.60 CPU-seconds (6.31%); matched checkpointed Mathlib costs 91.15/1,018.22 (8.95%). Their separate wall ratios are 9.26% and 14.67%. SubexponentialSAT primary accounting reports 3,399.30 checker CPU-seconds for all 14,721 accepted modules, or 7.25% of complete source work. Using only the 3,977 new targets' source cost gives 15.79% for the same whole-closure checker numerator. This denominator change illustrates why a single ratio cannot stand for every boundary. Module-local references agree on 14,401/14,721 modules; historical repaired `e0c92d51/3fe83c5a` replays agree, too. The other 320 lack module-local references.

3.2 Completed work and its limits

162,170 of 163,096 modules completed (99.43%); 926 were deferred. Successful work is 168.956 P / 302.399 E CPU-hours. Retired intervals total 31.529 wall-hours, including drain. The calendar span is 33.086 hours, including 1.557 hours of gaps. Final native failures / dependency blocks: 0/0; 66 unsuccessful attempts are excluded from successful CPU.

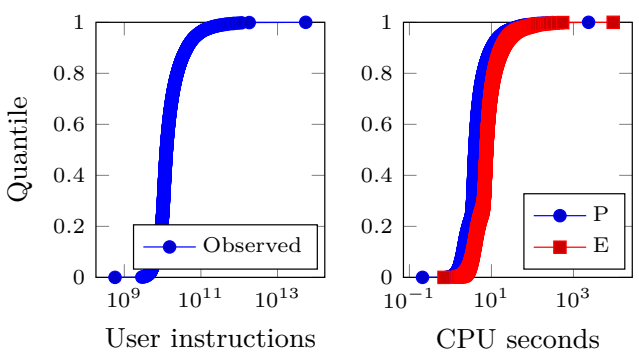


Figure 2: Distributions for 162,170 completed modules. User-mode instructions were selected after the failed gate; CPU curves separate P/E. Dependency order and user deferral prevent extrapolation.

Interval 1 (128,331 modules): 134.651 P / 256.668 E CPUh, 24.625 wallh. Interval 2 (33,788 modules): 32.718 P / 45.731 E CPUh, 6.179 wallh. Interval 3 (51 modules): 1.587 P / 0.000 E CPUh, 0.726 wallh. Intervals 2–3 share the memory-admission rule; cohort changes prevent a causal throughput comparison.

User-mode instructions total 5.276×10^{15}; the per-module median and 95th percentile are 1.258×10^{10} and 7.851×10^{10}, respectively. Figure 2 shows their distribution; OS user-mode instructions do not attribute Lean-kernel work.

Marked/unmarked: 5,848/156,322 completed modules; 30.226/138.730 P CPUh, 55.476/246.923 E CPUh. Outside/inside the default graph: 32,261/129,909 completed modules; 30.882/138.074 P CPUh, 56.237/246.162 E CPUh. These partitions cover the same observed set. Markers are source proxies, not kernel CPU phases; full-frame coverage has a separate denominator.

Dependency order and exclusion of unfinished high-memory families bias these observations; earlier family successes remain included. Missing work may cost more. Neither coverage, the 42 samples nor a model fills it. P/E work is a separate lower-bound vector for this per-module protocol, not a converted whole-library estimate. Module-wall sums are not machine wall.

External cost comparison. The earlier report builds 122,458 modules at `fd4aeeb2` on 32 cloud vCPUs in 13.8 wall-hours [10]. Its approximate 410 CPU-hours include 330 compilation hours extrapolated from 26 quiet laptop modules (±30%), plus 80 Comparator hours. Different revisions, hardware, scopes and our post-hoc criterion prevent calibrated agreement. Its 26-module kernel-skipping ablation and roughly 30,900 cloud profiles remain build-side phase precedents.

4 ProofServe design

ProofServe turns the cost and coverage findings into separate `check`, `audit` and compatibility-only `serve` interfaces. Its original-kernel route preserves mathematical checking rules while adding explicit coordination and artifact trust.

4.1 Check and resume a dependency closure

A checked prefix amortizes preparation while preserving each module's declared dependency context. `proofserve check <project>` discovers compiled local roots or uses explicit roots, traverses imports, and reports missing artifacts. The checker builds a graph from ordinary declarations' referenced constants. Ready declarations run against a checked frontier using the original kernel operation; the coordinator validates returned identities and commits the next frontier after all workers in a wave join. Nonordinary declarations and their ordinary prerequisites use the original replay path before the remaining graph runs. Unsupported local structure retains conservative fallback.

Review-driven repairs and theorem-body policy. Two read-only reviews of `e0c92d51` and `2628d727` traced coordinator defects; implementation-line regressions reproduced shadowed inputs, omitted axioms, unsafe duplicates, proof-body context substitution and malformed Quot restoration. Earlier O execution controls also exposed cross-project certificate reuse and added bad-proof parts. Repairs bind inputs, validate original contexts and quotient tags, separate worker events and preserve nonzero-exit dominance. The proposed project-IR stdout-forgery route remains unconfirmed. The issues have regression repairs; the repaired binary has no independent runtime review.

The frozen checker is `e000ed62`, replacing `e1557c77`. Its default *shared-checked* policy retains a kernel-checked theorem body when a later theorem has the same interface. A crafted serialized proof that passes only by depending on another module's theorem body can therefore be accepted while official checking rejects it in its declared import context. The accepted proposition is still derived from kernel-checked proofs in the shared checked closure and its reported axioms; this is a different artifact-context guarantee, not original-proof replay parity.

For matching theorem-body context behavior, use `--strict-theorem-bodies`, which isolates distinct bodies. Both modes explicitly check duplicate bodies skipped by upstream replay; temporary checking names enter neither the shared frontier nor the axiom set. Receipts and checkpoint identities bind the policy; ABI 6 invalidates old state. Strict-mode CPU and wall costs

must come from the final matrix, not historical e155 isolation runs.

Closed and some higher-order native evaluations remain unsupported; official default/fresh and operational exit conventions also differ. Editable state and arbitrary malformed metadata remain trust limits. Reports and controls are in `packaging/checker-review-*.md` and `checker/results/theorem-body-policy/`.

Disabling persistence changes Nat.Basic/Grid wall by +3.3/+5.2% and CPU by +12.1/+16.9%. Disabling split fallback changes Nat.Basic/Grid wall by +20.0/+12.4% and CPU by -1.4/-1.4%. These are eight-worker comparisons of the frozen tool, not historical prototype ratios. Persistent marking occurs between joined waves; both disabling flags remain available.

Completed segments serialize their successful prefix for reuse. Toolchain, artifacts, Lake lock and checkpoint configuration must match; a directory lock protects a live invocation. An interrupted segment remains incomplete. Receipt v1 distinguishes the accepted prefix, current-invocation CPU and previously completed segment CPU, so a cache hit is identifiable. It records Lean and dependency versions, module-set hash, accepted and rejected modules, reported axiom sets and CPU/wall clocks. A restart experiment charges restoration and restart overhead.

4.2 Audit coverage and explain trust

Audit connects source inventory, the default import graph and the actual checking receipt. `proofserve audit <project>` masks comments and strings before counting kernel-decision sites and extracts imports. Supported declarative Lake configuration gives default-root coverage; executable or unsupported configuration yields an explicit unknown unless evaluated roots are supplied. Supplied roots retain their supplied status. The receipt contributes its checked compiled closure and reported axioms; a partial receipt reports the last successful prefix. These lists enumerate axiom declarations in accepted modules, rather than minimal per-theorem dependency sets. Per-module costs require explicit source-bound observations.

The tool trusts Lean's kernel plus its artifact reader, context metadata, checked-token FFI, quiescent marking and editable local checkpoints. Recorded file identities are noncryptographic; receipt reuse is not remote attestation or a proof of checkpoint integrity. Internal symbols and serialized metadata require a matching SDK. Unknown toolchains fail clearly or explicitly select the matching official checker, identifying that path and its scope. The current receipt lacks a binding to the current source digest; audit exposes this gap. Thus acceptance, source coverage and reported axiom declarations remain separate answers.

Closure/mode	Clock	1	2	4	8
Nat.Basic D	CPU s	110.25	111.29	112.13	113.45
	Wall s	110.81	82.73	69.69	65.31
Nat.Basic S	CPU s	–	–	–	283.78
	Wall s	–	–	–	276.05
Grid D	CPU s	751.16	778.64	799.13	835.76
	Wall s	753.99	517.29	398.37	352.35
Barrier D	CPU s	732.64	–	–	766.34
	Wall s	735.59	–	–	425.91

Table 1: Medians (s), workers in columns, three complete repeats. D: shared-checked; S: strict. Full ranges are in the artifact; Grid strict is censored, reported only in the text.

5 Evaluation: performance and verification coverage

The frozen checker is `e000ed62`; the original driver is `cc5bcc18`, and publication `818f7422` records the six-hour collection. Native cell costs include the closed cgroup's whole process tree. Protected `zettai-dev` work remained running; about one host P-core was observed before launch. This is not exclusively idle-host performance.

5.1 RQ3: original-kernel scaling and alternatives

Nat.Basic/Grid/Barrier check 1,330/3,793/4,449 closure modules; the overlapping counts are not distinct coverage. Table 1 uses three fresh-state repeats per completed point. Barrier 2/4-worker cells were cut to fit the six-hour limit. Eight versus one worker gives 1.70/2.14/1.73× wall speedups for Nat.Basic/Grid/Barrier, with CPU increases of 2.9/11.3/4.6%, respectively. Nat.Basic strict mode costs 4.23× wall and 2.50× CPU versus default at eight workers. Grid strict mode timed out at 1,800 s (at least 5.1× default-eight median); this single censored attempt is not a strict median or full cost.

The 71-cell plan has 61 returned attempts and 10 unattempted cells. Unrun cells are Grid strict r2–3 and Grid/Barrier official default/fresh r2–3 (eight). Nat.Basic official fresh r3 was attempted but stopped at 77.70 s by the total window, not its per-cell cap. No partial observation fills a completed-run median.

All nine complete official samples accept their requested roots: Nat.Basic default/fresh 3/2, Grid and Barrier 1/1 each. Only Nat.Basic default has a three-repeat median (407.82 s wall). Stock default trusts imports and checks local declarations; fresh replays the final imported constant map [4]. Its search-path discovery and checking ledger differ from ProofServe's per-module work; these are not whole-ledger speedup comparisons. Prior nanoda/Lean timing (median 0.21, four threads

versus one, export excluded, 401/416 completed) is not our speed baseline [10, 9].

The existing bvar, unknown and sort controls all produce original-kernel rejection messages. One deliberate interruption and resume preserves the 1,330-module result and reuses one segment. Combined wall/CPU is 81.50/124.11 s, 16.94/10.68% above uninterrupted four-worker medians. Within each dataset, all 45 complete fixed native points and the resumed point have identical input identities, accepted sets, declared axiom lists and module hashes. The 15 declared axioms include `sorryAx` and compiler/runtime axioms: an inventory, not a sorry-free or minimal-use certificate.

Supplementary library checking. Built-subset checking has not yet returned a scope. Deferred OAI has no compiled artifact, so its capacity precheck is unavailable, not a proof rejection. Report only the verified prefix, with accepts/input frame, attempted roots, rejects, operational stops, unattempted scope, pins and official references. No partial prefix establishes whole-source checking or parity.

5.2 RQ4: default targets miss a third of decision sites

The full-frame import graph misses 19.78% of source files and 33.30% of lexical kernel-decision sites at our corpus pin. It covers 130,834 of 163,097 source units (including configuration), leaving 32,263 outside; the omitted files contain 451,008 of 1,354,379 sites. The earlier report finds 30,697 files outside its default graph, about 25% of its 122,459-node source inventory including the root [10]. The revisions and root-count conventions differ, but both demonstrate that a successful default build covers a subset of the source tree. The 33.30% figure describes decision sites, not module count.

Lexical density makes the workload difference visible even before timing. After masking comments and strings, the AI corpus contains 1,354,379 kernel-decision sites in 5,848 files; the pinned Mathlib subtree has seven sites in four of 8,529 files. Normalizing by decimal source megabytes gives a 10,668-fold density ratio. An audit should expose these labels alongside measured costs and uncertainty, rather than convert a site count into CPU work.

Repeated verdicts provide a second, separate kind of evidence. A historical panel checks 2,524 eligible stored-proof targets from three modules at 1, 2, 4 and 8 workers, three repeats each: 30,288 accepted checks and 22,716 one-worker/higher-worker comparisons. Verdicts, diagnostics and allocation counts agree in every observed pair, while elapsed time varies. These fixed-panel observations complement the installed-package rejection controls below.

Table 2 uses relocated tool `818f7422`, native `e000ed62` and Lean 4.34.1 (data `0516279d`). Newspeak/Solar/AI

Project	N/O modules	ProofServe CPU	ProofServe Wall	Official CPU	Official Wall
Newspeak	754/105	109.06	110.64	38.64	38.74
Solar	2368/12	167.21	169.32	5.51	5.54
AI proofs	10753/9	2481.04	2579.75	37.38	40.99

Table 2: Final installed-tool costs in seconds: one fresh run per arm, one worker. N/O modules are native full closure / official local roots. CPU and wall are separate. All 126 common roots pass both; unequal checking obligations provide no equal-obligation speedup comparison.

inputs are `e2232d8f/f4f5c9a1/3042904c`. The AI set is `samuelburbulla/proofs`: an upstream collection-level AI declaration, with model, AI fraction and human-editing fraction undisclosed. All nine native/official/audit commands complete uninterrupted; native checks use fresh checkpoints. Official default trusts imports; common-root agreement establishes neither whole-closure nor axiom-set parity.

Each arm has one observation, in fixed project and native/official/audit order, without cache flushes or confidence intervals. Cell costs exclude packaging. CPU is completed-command wait4 user+system including reaped descendants; wall is monotonic time. Background workloads remain running. The AI cgroup records 888 memory.max events, no OOM kill or resource stop: these are resource-bounded, not unconstrained-memory costs.

Audit covers Newspeak 105/105, Solar 12/32 and AI 9/9 local files. Solar's 20 omitted files feed a separate, unexecuted EVM checking flow. Newspeak reports a project `native_decide` axiom; declared inventories assign no per-theorem axiom use or sorry-freedom. Exact roots, inventories and records remain in `data/external-projects/e000ed62-final/`.

Historical dirty-packaging preview controls, including offline-container and rejection tests, remain separate from final timing and independent runtime review. The final local installation records glibc 2.41; no new portability test or service evaluation is claimed.

Artifact and reproduction. The artifact is at https://github.com/yunwei37/proofserve. Use `proofserve check /path/to/project` and `proofserve audit /path/to/project`; build with `cd checker; lake build`. `docs/paper/generate_census.py` and `docs/paper/generate_system.py` consume committed CSVs. Generation retains deferred identities and unsuccessful attempts.

6 Limitations and related work

Selected cost differences do not define a universal two-class law. The dependency-ordered compilation prefix does not establish representativeness; its missing tail

includes memory-intensive families. The original two-draw strata and fragile heavy-tail uncertainty cannot fill that gap. Lexical markers are proxies, not runtime frequencies. Selected closures overlap, module-local replay trusts imports, and target windows exclude shared preparation. Cross-protocol ratios, cache/order and hardware effects limit generalization; historical Subexponential-SAT module-local reference coverage is 14,401/14,721. Failed and blocked modules have no successful cost; their exclusion from cost distributions is explicit. Dynamic phase weights remain unmeasured.

Finite verdict agreement does not prove arbitrary-extension safety, future budget sufficiency or soundness of the coordinator. Reported axiom unions retain their actual scope: acceptance can depend on axioms, unsafe/partial exclusions remain visible, and semantic statement faithfulness requires separate review. Current source binding, malicious checkpoint integrity, executable Lake configuration and uncompiled/generated sources require explicit additional evidence. Performance remains bounded to these closures and background conditions; strict Grid and official repeats are incomplete. Final external timings have one observation per arm and fixed order, with common-root rather than full-closure official parity; the AI disclosure is an upstream collection-level attribution, not an independent authorship audit.

Parallel proof checking and asynchronous tasks have precedents in Isabelle, Coq and Lean [13, 1, 3]. Independent checkers and Lake's shipped workflow already address rechecking [7, 6]; the same-corpus tooling report also proposes comparable outcome, timing and export-hash records [10]. Kimina and AXLE supply verification services [12, 14]. Our contribution is the measured boundary between checking and recomputation and its integration into a usable original-kernel tool, rather than new parallelism, receipt or serving mechanisms.

7 Conclusion

Matched measurements distinguish cheap proof checking from certificate recomputation. Observed compilation yields biased-subset P/E lower bounds, not a full-library estimate. ProofServe exposes scope through check, audit and receipts; parallelism saves wall time at higher CPU cost. Default coverage and repeatability establish different facts. The repaired coordinator still needs independent runtime review; external costs retain their unequal checking scopes.

References

[1] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[2] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[3] Lean contributors. Lean 4.19.0: Parallel elaboration, 2025.

[4] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026.

[5] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026.

[6] Lean contributors. Lean 4.35.0 release notes, 2026.

[7] Lean contributors. lean4checker and the built-in leanchecker, 2026.

[8] Lean FRO contributors. The lean fro year 4, part 1 roadmap, 2026.

[9] Nanoda contributors. Nanoda: external lean type checker, 2026. Independent kernel implementation.

[10] nealmcb. Verifying openai/math in lean: cost and checker study. Technical report, revision c9570a82, 2026. Full report inspected 3 November 2026.

[11] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[12] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[13] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009.

[14] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.